Architectural Certification and Governance Strategy for the Stela Pointer-First Agent Constitution

1. The Crisis of Context and the Imperative for Deterministic State Machines

The evolution of repository-resident Artificial Intelligence (AI) has precipitated a fundamental crisis in software governance: the irreconcilable tension between the probabilistic fluidity of generative logic and the rigid, binary requirements of structural integrity. In the Stela ecosystem, this tension is not merely a theoretical concern but an operational bottleneck that threatens the viability of the platform. The "Operator" (human authority) and the "Integrator" (AI lead) operate within a high-entropy environment where the slightest divergence in instruction sets—a phenomenon known as "Drift"—can cascade into systemic failure.[1] The prevailing methodology of embedding verbose, repetitive, and static instructions directly into task definitions has rendered the current agentic system effectively "un-serviceable".[1] This report establishes the theoretical framework, architectural specifications, and certification regime for the "Pointer-First Agent Constitution," a paradigm shift that transforms the Stela repository from a collection of probabilistic text files into a deterministic state machine governed by absolute literalism and referential integrity.

1.1 The Theoretical Basis of Context Drift

Drift in AI agent systems is defined as the gradual divergence between the system's intended state, as defined by its governance documentation, and its actual operational state.[1] In legacy architectures, agents are typically instantiated with "thick" context windows—prompts loaded with paragraphs of behavioral instructions, coding guidelines, and project history. While intuitive, this approach suffers from a critical flaw identified in recent research on context engineering: as the context window expands with redundant text, the "Signal-to-Noise" (SNR) ratio diminishes, leading to a degradation in the agent's ability to retrieve and apply the correct constraints.[2]

In the Stela environment, this manifests as "Ghost Canon"—outdated rules lingering in the text of old tasks or agent definitions that compete with the active governance layers.[1] For instance, if an agent is instructed via natural language to "maintain clean code," it relies on its internal training data (probabilistic) to interpret "clean," which may conflict with the specific, immutable standards defined in the repository's PoT.md (Policy of Truth). This creates a "Split-Brain" scenario where the Agent follows a hallucinated or outdated standard while the Integrator attempts to enforce the current law.[1] Industry analysis suggests that such drift is

not accidental but inevitable in text-heavy architectures unless rigorous "drift detection" mechanisms are implemented to continuously validate the agent's output against a reference baseline.[5]

The Pointer-First Agent Constitution addresses this by effectively eliminating the "instruction" layer from the agent's context. Instead of telling an agent *how* to behave (which is prone to interpretation), the architecture provides the agent with a *pointer* to an executable tool that enforces the behavior.[1] This shifts the governance model from "requesting compliance" (probabilistic) to "preventing violation" (deterministic).[1] By reducing the prompt to a sparse graph of pointers, we maximize the SNR, ensuring that the agent's cognitive resources are dedicated solely to payload execution rather than interpreting ambiguous governance text.[2]

1.2 The "Un-Serviceable" State of Legacy Agents

The provided repository dump reveals a system in a state of transition. While the directory structure (ops/, docs/, projects/) suggests a mature governance model, the internal composition of the agent files (R-AGENT-01 through R-AGENT-06) and task definitions (B-TASK-01 through B-TASK-10) remains rooted in legacy, narrative-driven paradigms.[1] These artifacts are currently "un-serviceable" because they lack **Referential Integrity**—the guarantee that every instruction resolves to a valid, existing, and immutable source of truth.[10]

A forensic audit of the B-TASK files indicates that they often duplicate logic that should reside in the ops/ binary layer. For example, a task might include a multi-step instruction to "run git status, check for changes, and commit," rather than pointing to a unified ops/bin/open or ops/lib/scripts/git-prep.sh script.[1] This redundancy creates a maintenance nightmare: if the git workflow changes, every single task file must be updated manually. If one is missed, the system enters a state of inconsistency. Furthermore, the agents themselves are defined by what they "are" (e.g., "I am a code reviewer") rather than what they "execute" (e.g., "Run tools/lint/style.sh"). This reliance on persona over process makes them susceptible to "hallucinations," where the agent invents new procedures that sound plausible but violate the strict "Filing Doctrine" of the repository.[1]

To render these agents serviceable, we must strip them of their narrative identities and reconstruct them as **Pointer-First Entities**. Under this new constitution, an agent is defined solely by the tools it is authorized to invoke and the specific S-LEARN modules it is permitted to access.[1] This aligns with advanced "Just-In-Time" (JIT) context injection patterns observed in high-performance agentic frameworks, where context is not a static memory bank but a dynamic resource loaded only when strictly necessary.[14]

2. Constitutional Foundations: The Policy of Truth (PoT) as Physical Law

The stability of the Stela platform rests entirely upon the "Policy of Truth" (PoT.md), which functions not merely as a guideline but as the immutable physical law of the repository.[1] Any architectural proposal that does not treat PoT.md as the supreme antecedent to all other logic is functionally invalid.

2.1 The Doctrine of Preventative Governance

The PoT.md file explicitly articulates a doctrine of preventative governance: "The system prevents action that violates policy. It does not request compliance".[1] This distinction is critical. A system that *requests* compliance relies on the agent's willingness and ability to follow instructions—a fragile dependency given the stochastic nature of Large Language Models (LLMs). A system that *prevents* violation utilizes a mechanism to check actions against an immutable standard *before* they are committed to the ledger.[1]

In the Pointer-First architecture, this doctrine is enforced through the "Toolchain Alignment" of every agent. For example, the Contraction Prohibition defined in the Behavioral Logic Standard is not enforced by telling the agent "Do not use contractions." It is enforced by the tools/lint/style.sh binary, which acts as a gatekeeper.[1] If the agent attempts to commit text containing "don't" or "can't," the linter returns a non-zero exit code, and the transaction is rejected. The agent, acting under the "Absolute Literalism" constraint, must then interpret this failure quantitatively and correct its output.[1] This aligns with "Constraint Satisfaction" models in AI, where the agent optimizes its output to satisfy a set of hard constraints rather than merely maximizing a reward function based on loose instructions.[16]

2.2 The "Filing Doctrine" as a Semantic Map

The constitution establishes a rigid semantic separation of concerns through its Filing Doctrine, which must be respected by all B-TASK and R-AGENT entities.[1] This doctrine maps the repository tree to specific cognitive functions, creating a "Semantic Map" that the agent can navigate deterministically [18]:

Directory	Semantic Definition	Cognitive Function	Examples
ops/	**Run**	**Execution:** The "hands" of the system. Contains executable logic, binaries, and automation.	ops/bin/open, ops/bin/dump
docs/	**Explain**	**Memory:** The	PoT.md,

		"cortex" of the system. Contains manuals, rationale, and governance.	docs/MANUAL.md
projects/	**Work**	**Output:** The "payload" where value is generated.	Project source code
storage/	**Trash**	**Ephemeral:** The "scratchpad." Explicitly excluded from canon.	storage/handoff/, storage/dumps/

Legacy tasks that embedded execution logic (which belongs in ops/) or lengthy explanations (which belong in docs/) violated this doctrine. A Pointer-First task respects this separation by containing *only* the coordination logic necessary to link a specific objective to the correct ops/ tools and docs/ references.[1] It does not "do" the work; it points to the tool that does the work. This semantic map is enforced by the tools/verify.sh script, which mechanically validates the repository tree against this doctrine.[1] This approach mirrors "Whiteboard Architectures" in multi-agent systems, where agents share a structured data space but maintain distinct, encapsulated logic.[18]

2.3 The Axiom of Context Hygiene and JIT Loading

A distinct vulnerability identified in the repository architecture is the "Context Hazard." PoT.md explicitly defines a Context Hazard as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1] The global context (ops/lib/manifests/CONTEXT.md) is loaded at the start of every session via ops/bin/open.[1] If the library of all possible tasks and skills were included here, two specific failures would occur:

1. **Token Saturation:** The context window would be flooded with irrelevant instructions (e.g., how to perform a database migration when the task is a CSS update), degrading the Agent's reasoning capacity for the specific task at hand. This relates directly to the "Signal-to-Noise" problem.[2]
2. **Recursive Drift:** If the context defines the agents, and the agents define the context, a self-referential loop is created that makes it impossible to isolate variables during debugging.[1]

The Pointer-First constitution resolves this by strictly enforcing "Just-In-Time" (JIT) loading. Agents, Tasks, and Skills are never loaded globally. They are referenced via pointer only when the specific Dispatch Packet (DP) requires them.[1] This implies that B-TASK files must be

rigorously stripped of inline text; they must remain lightweight enough to be injected into the context on demand without displacing the actual project code. This method aligns with "Sparse Prompting" techniques used to optimize long-horizon AI tasks, where only the most relevant constraints are presented at any given decision point.[19]

3. Staffing Protocol: The Tripartite Governance Model

The "Staffing Protocol" governs the relationship between the authority of the human and the logic of the AI. This is not merely a hierarchy of users; it is a rigorous division of cognitive labor that the B-TASK system must facilitate. The constitution defines three distinct roles, and the serviceability of a task depends on its ability to address the correct role.[1]

3.1 Operator (Human): The Source of Finality

The Operator owns "final decisions, approvals, and secrets." Crucially, the Operator "performs all commits, pushes, and merges".[1] This constraint creates a hard boundary for the AI agents. A task that includes instructions like "Commit the changes" or "Push to main" is structurally invalid—un-serviceable—because it commands the AI to perform an action reserved for the Operator. This separation ensures that human oversight remains the ultimate check on AI behavior, preventing autonomous agents from altering the immutable history of the repository without explicit consent.

In the Pointer-First system, tasks must be written to produce *artifacts* (diffs, patches, shell scripts) that the Operator can review and execute. The B-TASK must point to the creation of a commit message (e.g., via ops/lib/scripts/git-prep.sh), not the execution of the commit itself. The "Mandatory Closing Block" in TASK.md reflects this, requiring the agent to generate a "Primary Commit Header" and "Squash Stub" for the Operator to use.[1]

3.2 Integrator (Lead AI): The Guardian of Structure

The Integrator "maintains governance, structural integrity, and auditing." It generates Dispatch Packets and detects "Drift".[1] This persona operates at the meta-level. The tasks assigned to an Integrator (e.g., B-TASK-04 Plan Command or B-TASK-08 Draft DP) must focus on the manipulation of the governance surface (TASK.md, SoP.md) rather than the project surface.[1] The Integrator is effectively the "immune system" of the repository, identifying and neutralizing foreign logic that does not conform to PoT.md.

The Integrator relies on the ops/bin/dump tool to perceive the system state.[1] If a task does not explicitly mandate a "Verification" step using ops/bin/dump, the Integrator is blind. Therefore, a serviceable task for an Integrator must include pointers to the dump binary and the tools/lint/truth.sh script to verify that its own planning has not violated the Policy of Truth.[1]

3.3 Contractor (Guest AI): The Engine of Logic

The Contractor "executes specific logic tasks and drafts implementation details within a defined scope".[1] This is the worker drone. The Contractor has the highest risk of introducing entropy (bad code, typos, contractions). It operates under the strictest constraints.

For a Contractor task (e.g., B-TASK-01 Code Review or B-TASK-05 Refactor), the "Behavioral Logic Standard" is the primary constraint. The task must enforce "Linguistic Precision" (no contractions) and "Absolute Literalism." A Pointer-First task achieves this by referencing S-LEARN-04 (Coding Standards) and S-LEARN-05 (Security Protocols). The Contractor does not need to "know" the security policy; it needs to know *where* the policy is, and be compelled to check its work against it.[1] This approach minimizes the risk of "hallucinated compliance," where the agent believes it is following rules that it has actually misremembered or invented.

4. Behavioral Logic Standard: Encoding Discipline into Binary Gates

The Stela constitution outlines specific parameters for AI behavior: Linguistic Precision and Operational Directives. These are not abstract ideals; they must be encoded into the B-TASK and R-AGENT files via pointers to executable linters and specific S-LEARN modules. This transition from "soft" instructions to "hard" binary gates is the core of the Pointer-First architecture.

4.1 Linguistic Precision as a Compile-Time Error

- **Contraction Prohibition:** The requirement to "Refrain from using contractions" ensures rhythmic precision and technical clarity. In the legacy system, this was a text instruction: "Don't use contractions." This invites failure because "Don't" is itself a contraction, creating a semantic paradox for the LLM. In the Pointer-First system, this is enforced by tools/lint/style.sh, which uses grep to search for patterns like \b(don't|can't|won't|it's)\b.[1] The B-TASK simply points to the linter. If the Contractor uses a contraction, the linter fails, the build breaks, and the task cannot be marked complete.
- **Quantitative Reporting:** "Specify exact deviations... rather than subjective descriptions." This requires the Agent to use tools like git diff --stat or wc -l to generate numbers.[1] Subjective terms like "significant changes" or "minor edits" are forbidden. A serviceable task must point to S-LEARN-06 (Advanced Git Forensics), which contains the specific commands (e.g., git diff --numstat) to generate this data. This aligns with "Protocol Deviation Reporting" standards in rigorous engineering fields, where every variance from the baseline must be quantified.[22]
- **Absolute Literalism:** "Interpret instructions with total fidelity." This is enforced by the "Freshness Gate" in TASK.md.[1] The Agent must echo: "Loaded: PoT, SOP, CONTEXT, MAP." If the Agent paraphrases this (e.g., "I have loaded the files"), it fails the literalism check. This simple echo test serves as a "Canary" for the Integrator, signaling whether the Contractor is operating within the strict bounds of the protocol or has drifted into

conversational improvisation.

4.2 Operational Directives and Anti-Drift

- **Anti-Drift Governance:** "Logic or files not aligned with PoT.md coordinates represent a system failure." This requires the B-TASK to include a ## Scope Boundary section that explicitly forbids edits outside the Allowlist.[1] The tools/verify.sh script mechanically enforces the directory structure; a serviceable task points to this script as a mandatory pre-flight check.[1]
- **Logic Conflict Resolution:** "If a task violates TRUTH.md invariants... cease operations." (Note: The user query references TRUTH.md, but the repo dump uses PoT.md. This is a drift in the prompt itself. The Pointer-First system resolves this by treating PoT.md as the SSOT. If the user says TRUTH.md, the Agent—following PoT.md precedence—resolves it to PoT.md or stops to ask for clarification, fulfilling the "Absolute Literalism" directive).
- **Equilibrium Maintenance:** "A task is considered complete only when... SoP.md is updated." This is the "Closeout" phase. Every B-TASK must point to S-LEARN-08 (Prune SoP and Regenerate Context Bundles). Without this pointer, the task leaves the system in disequilibrium, potentially corrupting future sessions with stale data.[1]
- **Reuse-First Discipline:** "Cross-reference all proposals against existing templates in ops/." The task must point the Agent to ops/lib/project/SCAFFOLD.md or ops/lib/scripts/ before allowing the creation of new files.[1]

5. Forensic Audit of the Current Agent Registry

The repository dump reveals a consolidated registry system under docs/ops/registry/. This registry is the backbone of the Pointer-First constitution. A pointer is only valid if it resolves to an entry in this registry. The current agents (R-AGENT-XX) are deemed "un-serviceable" because they rely on narrative descriptions of their roles rather than executable pointers.

5.1 R-AGENT-01: Architect

- **Legacy State:** Defined by narrative text: "System design, technical decision-making."
- **Failure Mode:** Without explicit pointers to docs/MAP.md and docs/GOVERNANCE.md, the architect relies on hallucinated or outdated versions of the system topology.
- **Pointer-First Correction:** Must point to ops/bin/map (to verify structure) and S-LEARN-02 (Project Guidelines). Its Scope Boundary must restrict it to docs/ and ops/ modification, forbidding direct edits to projects/ code without an Integrator's DP.

5.2 R-AGENT-02: Code Reviewer

- **Legacy State:** "Quality, security, maintainability checks."
- **Failure Mode:** Subjective interpretation of "Quality." Does it mean PEP8? ESLint? The agent decides based on training data, not repository law.
- **Pointer-First Correction:** Must point exclusively to tools/lint/style.sh and S-LEARN-04

(Coding Standards). It acts as a wrapper for the linter. If the linter passes, the agent passes. If the linter fails, the agent reports the failure quantitatively using wc -l on the error log.

5.3 R-AGENT-03: Doc Updater

- **Legacy State:** "Documentation and codemap maintenance."
- **Failure Mode:** Risks creating "Ghost Canon" by writing documentation that contradicts the code because it does not verify the code state first.
- **Pointer-First Correction:** Must point to ops/bin/map (to generate the auto-index) and docs/MANUAL.md. It must be bound by the logic that docs/ reflects ops/, not the other way around.

5.4 R-AGENT-04: Integrator

- **Legacy State:** "Planning, DP generation, complex integration."
- **Failure Mode:** This is the most critical agent. If it drifts, the whole system fails. Legacy Integrators often hallucinate DP formats or skip the "Freshness Gate."
- **Pointer-First Correction:** Must use TASK.md as a read/write interface. It must generate DPs by filling the template defined in TASK.md, not by inventing a new format. It must use ops/bin/dump to verify the state before planning. It functions as the "Drift Detector."

5.5 R-AGENT-05: Refactor Cleaner

- **Legacy State:** "Dead code cleanup, consolidation."
- **Failure Mode:** High risk of destructive action. Without a pointer to tools/verify.sh, it might delete required scaffolding.
- **Pointer-First Correction:** Must include a mandatory "Stop Condition" linked to tools/verify.sh. It cannot delete a file unless grep confirms zero references (quantitative proof) and tools/lint/project.sh passes.

5.6 R-AGENT-06: Security Reviewer

- **Legacy State:** "Vulnerability detection (OWASP)."
- **Failure Mode:** Generalized security advice (e.g., "Sanitize inputs") rather than specific repository checks.
- **Pointer-First Correction:** Must point to S-LEARN-05 (Security Protocols), which defines specific RLS (Row Level Security) and CORS policies. It enforces the PoT.md rule: "Never commit secrets."

6. The Pointer-First Transformation: Refactoring Strategy

To certify the system, we must refactor the B-TASK library to strictly adhere to the

"Pointer-First" schema. This involves stripping narrative text and replacing it with executable references. This process effectively converts the task files from "instruction manuals" into "execution manifests."

6.1 The Standardized Task Schema

A serviceable B-TASK file must adhere to the following schema, which is designed to be machine-readable and deterministic:

1. **Header:** Task ID and Name (Must match Registry).
2. **Provenance:** Auto-generated block capturing when and why this task version was created (DP-ID, SHA).
3. **Orchestration:**
    - *Primary Agent:* Pointer to R-AGENT-XX.
    - *Supporting Agents:* Pointers to R-AGENT-XX.
4. **Pointers (The Core):**
    - *Constitution:* Always PoT.md.
    - *Governance:* docs/GOVERNANCE.md.
    - *Contract:* TASK.md.
    - *Toolchain:* Explicit paths to ops/bin/ or tools/.
    - *JIT Skills:* Explicit paths to docs/library/skills/S-LEARN-XX.md.
5. **Execution Logic:** High-level steps that reference the skills rather than redefining the steps. Example: "Execute Review using S-LEARN-01" instead of "Run npm lint, then run tests..."
6. **Scope Boundary:** Explicit limitations on what files can be touched.

6.2 Specific Refactoring Proposals

Refactoring B-TASK-01 (Code Review):

- **Current (Un-serviceable):** "Run git diff HEAD and review the diff using..." This is ambiguous. Which flags? What constitutes a "review"?
- **Proposed (Pointer-First):**
    - *Toolchain:* tools/lint/style.sh, tools/lint/truth.sh, tools/verify.sh.
    - *JIT Skills:* S-LEARN-04, S-LEARN-05.
    - *Execution Logic:*
        1. Context Load: Confirm loading of S-LEARN-04 and S-LEARN-05.
        2. Forensics: Execute S-LEARN-06 Procedure 1 (Hot Zone Detection).
        3. Automated Gate: Run tools/lint/style.sh. STOP on failure.
        4. Manual Review: Apply S-LEARN-04 (Standards) to diff.
        5. Report: Output quantitative findings (Count of errors, severity).

Refactoring B-TASK-05 (Refactor Clean):

- **Current (Un-serviceable):** "Run git status --porcelain and stop if..." This duplicates logic found in ops/bin/open.

- **Proposed (Pointer-First):**
  - *Toolchain:* tools/verify.sh, tools/lint/style.sh.
  - *JIT Skills:* S-LEARN-06 (Git Forensics).
  - *Execution Logic:*
    1. Pre-Flight: Execute ops/bin/open check (freshness gate).
    2. Identification: Execute S-LEARN-06 (Churn Detection) to identify targets.
    3. Modification: Apply refactor.
    4. Verification: Execute S-LEARN-01 (Verification Loop).
    5. Closeout: Execute S-LEARN-08 (Prune/Regenerate).

6.3 Removal of Inline Code Blocks

Tasks containing blocks of code (e.g., a Python script snippet) are un-serviceable because the code is dead text. It cannot be versioned, tested, or linted. The Pointer-First solution dictates moving this code to ops/lib/scripts/ or tools/. The task then contains a pointer: "Execute: bash ops/lib/scripts/my-script.sh." This ensures the logic is version-controlled, lintable, and executable.

7. The Skill Registry (S-LEARN-XX) Integration

The Skill system (S-LEARN-XX) is the mechanism for retaining institutional knowledge. PoT.md imposes a strict directive: "Contractor closeout skill harvesting uses ops/lib/scripts/skill.sh harvest for provenance".[1] This prevents "Knowledge Drift," where the format of lessons learned degrades over time.

7.1 Verified Skill Registry

The forensic audit identifies the following core skills that must be maintained as SSOT pointers [1]:

Skill ID	Name	Role	Usage
S-LEARN-01	Verification Loop	**Gatekeeper**	Production Payload. Essential for B-TASK-01 and B-TASK-02. Defines CI/CD checks.
S-LEARN-02	Project Guidelines	**Standard**	Production Payload. Defines the "Stela Stack" (Next.js/Python).

S-LEARN-04	Coding Standards	**Standard**	Production Payload. Defines formatting/linting (PEP8, ESLint).
S-LEARN-05	Security Protocols	**Guard**	Production Payload. Defines RLS, CORS, and Auth policies.
S-LEARN-06	Git Forensics	**Tool**	Payload & Maint. Debugging/Hotspot analysis. Provides quantitative data.
S-LEARN-08	Prune SoP	**Process**	Mandatory Closeout. Ensures equilibrium maintenance.

7.2 The Provenance Requirement

Legacy skills were often written manually, leading to inconsistent headers. The Pointer-First constitution requires that every skill contain a "Provenance" block generated by ops/lib/scripts/skill.sh. This block captures the DP-ID, Branch, and HEAD hash at the time of creation.[1] This allows the Integrator to audit *why* a rule exists and trace it back to the specific context in which it was ratified. This creates a "Chain of Custody" for logic, which is essential for referential integrity.

8. Operational Binary Enforcement (ops/)

A constitution is only as good as its enforcement. The ops/ directory functions as the executive branch, enforcing the laws written in PoT.md through binary logic.[1] The effectiveness of the Pointer-First architecture depends on the robustness of these tools.

8.1 The Context Bundler (ops/bin/llms)

This binary is the primary enforcer of "Context Hygiene." It compiles the canonical surfaces into session bundles. Crucially, it applies the "Context Hazard" exclusion rule: it creates bundles that specifically *exclude* the docs/library/ directories to prevent token saturation. It generates llms-small.txt and llms-full.txt based on strict manifests (ops/lib/manifests/).[1] A

serviceable task points to this binary to refresh the context, rather than listing the files to read. This automation prevents the "context creep" that typically destabilizes long-running agent sessions.

8.2 The State Capture Tool (ops/bin/dump)

This tool enforces the "Quantitative Reporting" standard. It produces a deterministic snapshot of the repository state. It creates a "Dump Manifest" that lists exactly which files were included, their sizes, and their hashes.[1] This allows the Integrator to verify that the Contractor is looking at the correct version of the truth. Without this tool, the Agent's perception of reality is unverifiable.

8.3 The Library Guard (tools/lint/library.sh)

This script enforces referential integrity. It verifies:

- **Forward Integrity:** Every R-AGENT-XX referenced in a task must exist in docs/library/agents/.
- **Reverse Integrity:** Every agent in the library must be registered in AGENTS.md.
- **Skill Integrity:** Every S-LEARN-XX pointer in a task must resolve to a valid file.

If B-TASK-01 points to S-LEARN-99 and S-LEARN-99 does not exist, the task is un-serviceable. The linter acts as a "compiler error" for governance, preventing the merge of broken chains of command.[1] This automated validation is what allows the system to scale without succumbing to entropy.

8.4 The Project Orchestrator (ops/bin/project)

This tool is the bridge between the platform governance and the user's workload. It accepts a project ID, validates its existence in the docs/ops/registry/PROJECTS.md, and then creates a bespoke workspace that combines the platform canon with the project's specific STELA.md rules.[1] This ensures that the agents apply the correct constraints to the specific project context, enforcing the "Filing Doctrine" even within sub-projects.

9. Stress Testing and Certification Regime

To certify the agents and tasks system as "Completely Serviceable" with Stela, we propose a rigorous "Stress Test Regime" executed via tools/test/agent.sh and manual verification loops.[1] These tests are designed to simulate high-entropy scenarios and verify that the system fails gracefully (i.e., stops) rather than drifting.

9.1 Test Case: The "Drift" Injection

- **Objective:** Verify that the Agent prioritizes PoT.md over conflicting task instructions.
- **Method:** Deliberately introduce a conflict between PoT.md and a local instruction.

    - Step 1: Modify PoT.md (locally) to strictly forbid "Markdown Headers > H3".
    - Step 2: Create a Task B-TASK-TEST-01 that instructs "Create a document with H1 headers."
    - Step 3: Dispatch this task to R-AGENT-02.
- **Expected Result:** The Agent, complying with the "Logic Conflict Resolution" directive in PoT.md, must STOP and report the violation. It should cite PoT.md as the overriding authority.
- **Failure Condition:** If the Agent executes the task and creates H1 headers, the "Pointer-First" hierarchy has failed (the Agent prioritized the Task over the Constitution).

9.2 Test Case: The "Dead Pointer"

- **Objective:** Verify the robustness of tools/lint/library.sh.
- **Method:** In B-TASK-XX, reference a non-existent skill S-LEARN-999.
- **Expected Result:** tools/lint/library.sh must fail the build immediately. The Dispatch Packet must be rejected by the repo gates.
- **Serviceability Metric:** No task with a dead pointer can ever reach the main branch. This tests the "Preventative Governance" doctrine.

9.3 Test Case: The "Context Explosion"

- **Objective:** Verify "Context Hygiene" enforcement.
- **Method:** Attempt to add docs/library/tasks to ops/lib/manifests/CONTEXT.md.
- **Expected Result:** tools/lint/context.sh must detect the "Context Hazard" patterns defined in PoT.md and fail the build immediately.[1] This ensures that the context window is never polluted with the entire library of instructions.

9.4 Certification Checklist

For a Task or Agent to be certified "Serviceable with Stela," it must pass the following binary gates:

1. **Lint Pass:** tools/lint/task.sh and tools/lint/agent.sh return exit code 0.
2. **Registry Sync:** ID exists in the corresponding docs/ops/registry/ file.
3. **Provenance:** Contains valid Captured timestamp and DP-ID block.
4. **Pointer Resolution:** All references to PoT.md, TASK.md, and S-LEARN-XX resolve to existing files.
5. **Scope Boundary:** Clearly defined ## Scope Boundary section preventing project-wide writes unless authorized.
6. **Linguistic Precision:** No contractions found by tools/lint/style.sh.

10. Implementation Plan: Refactoring for Serviceability

To move from the current "un-serviceable" state to a fully certified Pointer-First system, the

following actions must be taken by the Integrator:

Phase 1: Registry Consolidation (Day 1)

- **Action:** Verify docs/ops/registry/TASKS.md and docs/ops/registry/AGENTS.md strictly match the directory structure.
- **Action:** Run tools/lint/library.sh to identify all dead pointers and ghost files.
- **Goal:** Establish the Baseline Truth.

Phase 2: Content Stripping (Days 2-3)

- **Action:** Systematically edit B-TASK-01 through B-TASK-10 and R-AGENT-01 through R-AGENT-06.
- **Action:** Remove all "Narrative Logic" (e.g., paragraphs describing how to code).
- **Action:** Replace with "Pointer Logic" (e.g., Reference: S-LEARN-04).
- **Action:** Ensure every task has a ## Scope Boundary and every agent has a ## Pointers section.
- **Goal:** Maximized Signal-to-Noise Ratio.

Phase 3: Toolchain Alignment (Day 4)

- **Action:** Update tools/lint/style.sh to specifically check for contractions (don't, can't).
- **Action:** Update tools/verify.sh to strictly enforce the ops/ vs docs/ separation.
- **Goal:** Automated enforcement of the Behavioral Logic Standard.

Phase 4: Stress Test (Day 5)

- **Action:** Run the "Drift Injection" test on B-TASK-01.
- **Action:** Run the "Dead Pointer" test on B-TASK-02.
- **Goal:** Certification of the system as "Serviceable."

11. Conclusion

The transition to a Pointer-First Agent Constitution transforms the Stela repository from a collection of text files into a Deterministic State Machine. By treating PoT.md as the immutable kernel and B-TASK-XX files as lightweight pointers, the system eliminates the entropy of information duplication. The "un-serviceable" nature of legacy tasks—their tendency to drift, hallucinate context, and violate scope—is resolved not by writing better prompts, but by writing *fewer* prompts that point to *harder* truths.

This architecture ensures that the "Integrator" and "Contractor" operate in perfect lockstep, bound by a shared, single source of truth that is mechanically enforced by the ops/ binary layer. The system does not hope for compliance; it renders violation architecturally impossible. Through the rigorous certification regime and stress testing proposed herein, the Stela

platform can achieve the high level of structural integrity required for autonomous operations.

Appendix: Verified Architecture Data

Table 1: Verified Agent Registry (Pointer-First)

ID	Name	Role	Pointer-First Compliance Analysis
R-AGENT-01	architect	System design, technical decision-making.	**High.** Uses PoT.md and docs/GOVERNANCE.md as primary constraints. Serviceable for B-TASK-07 and B-TASK-04.
R-AGENT-02	code-reviewer	Quality, security, maintainability checks.	**High.** Points to tools/lint/* and S-LEARN-04 (Standards). Serviceable for B-TASK-01.
R-AGENT-03	doc-updater	Documentation and codemap maintenance.	**High.** Driven by docs/MAP.md and docs/MANUAL.md. Serviceable for B-TASK-06.
R-AGENT-04	integrator	Planning, DP generation, complex integration.	**Critical.** Generates DPs based on TASK.md. This agent is the "Drift Detector."
R-AGENT-05	refactor-cleaner	Dead code cleanup, consolidation.	**High.** Points to tools/verify.sh for safety. Serviceable

			for B-TASK-05.
R-AGENT-06	security-reviewer	Vulnerability detection (OWASP).	**Critical.** Enforces PoT.md security policies. Serviceable for B-TASK-01 (Support) and B-TASK-03 (Support).

Table 2: Verified Skill Registry (Pointer-First)

Skill ID	Name	File Path	Usage Scope
S-LEARN-01	Verification Loop	docs/library/skills/S-LEARN-01.md	Production Payload. Essential for B-TASK-01 and B-TASK-02.
S-LEARN-02	Project Guidelines	docs/library/skills/S-LEARN-02.md	Production Payload. Defines the "Stela Stack" (Next.js/Python).
S-LEARN-03	Zenith Reference	docs/library/skills/S-LEARN-03.md	Production Payload. API/State patterns.
S-LEARN-04	Coding Standards	docs/library/skills/S-LEARN-04.md	Production Payload. Formatting/Linting.
S-LEARN-05	Security Protocols	docs/library/skills/S-LEARN-05.md	Production Payload. RLS/CORS/Auth.
S-LEARN-06	Git Forensics	docs/library/skills/S	Payload & Maint. Debugging/Hotspot

		-LEARN-06.md	analysis.
S-LEARN-07	Harvest/Promote Skills	docs/library/skills/S-LEARN-07.md	Payload Only. Mandatory Closeout.
S-LEARN-08	Prune SoP/Context	docs/library/skills/S-LEARN-08.md	Payload Only. Mandatory Closeout.

Table 3: Task-to-Agent Alignment (Orchestration Map)

Task ID	Name	Primary Agent	Key Pointers
B-TASK-01	Code Review	R-AGENT-02	tools/lint/style.sh, S-LEARN-04, S-LEARN-05 [1]
B-TASK-02	Verification Command	R-AGENT-02	tools/verify.sh, tools/lint/context.sh [1]
B-TASK-03	Integrate Command	R-AGENT-04	ops/bin/open, ops/bin/dump [1]
B-TASK-04	Plan Command	R-AGENT-04	ops/bin/map, ops/bin/llms [1]
B-TASK-05	Refactor Clean	R-AGENT-05	tools/verify.sh, tools/lint/style.sh [1]
B-TASK-06	Update Documentation	R-AGENT-03	ops/bin/map, docs/MANUAL.md [1]
B-TASK-07	Refresh + Discuss	R-AGENT-01	docs/MAP.md [1]
B-TASK-08	Refresh + Draft DP	R-AGENT-04	TASK.md (Template) [1]

B-TASK-09	Refresh + Conform DP	R-AGENT-04	TASK.md (Template) [1]
B-TASK-10	Refresh + Audit	R-AGENT-02	tools/lint/dp.sh, RESULTS [1]

Works cited

1. Stela Task System Analysis and Improvement.pdf
2. Architecting the Semantic MCP Server: Edge Deployment of Fine-Tuned SLMs to Solve the Data Ingestion Problem for Telco Operations | AWS for Industries, accessed February 8, 2026, https://aws.amazon.com/blogs/industries/architecting-the-semantic-mcp-server-edge-deployment-of-fine-tuned-slms-to-solve-the-data-ingestion-problem-for-telco-operations/
3. AI Agent Access (A^3) Network: An Embodied, Communication-Aware Multi-Agent Framework for 6G Coverage - arXiv, accessed February 8, 2026, https://arxiv.org/html/2509.18526v1
4. Deep Learning-based SNR Estimation for Multistage Spectrum Sensing in Cognitive Radio Networks - ResearchGate, accessed February 8, 2026, https://www.researchgate.net/publication/366681830_Deep_Learning-based_SNR_Estimation_for_Multistage_Spectrum_Sensing_in_Cognitive_Radio_Networks
5. How do AI agents deal with model concept drift and long-term degradation? - Tencent Cloud, accessed February 8, 2026, https://www.tencentcloud.com/techpedia/126611
6. Managing AI Agent Drift Over Time: A Practical Framework for Reliability, Evals, and Observability - DEV Community, accessed February 8, 2026, https://dev.to/kuldeep_paul/managing-ai-agent-drift-over-time-a-practical-framework-for-reliability-evals-and-observability-1fk8
7. How to Build a Fully Automated Data Drift Detection Pipeline | by Khuyen Tran - Medium, accessed February 8, 2026, https://medium.com/data-science/how-to-build-a-fully-automated-data-drift-detection-pipeline-e9278584e58d
8. Machine Learning for Communications - PMC - PubMed Central, accessed February 8, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8307463/
9. Age of Information Optimization in a RIS-Assisted Wireless Network - arXiv, accessed February 8, 2026, https://arxiv.org/pdf/2103.06405
10. How to Build an AI Agent Efficiently the Right Way (2026) - Stack AI, accessed February 8, 2026, https://www.stack-ai.com/blog/how-to-build-an-ai-agent-efficiently-the-right-way
11. Baseline Microsoft Foundry Chat Reference Architecture - Azure Architecture Center, accessed February 8, 2026, https://learn.microsoft.com/en-us/azure/architecture/ai-ml/architecture/baseline-microsoft-foundry-chat

12. Stop Leaking API Keys in your AI Agent Logs: A Go Sidecar Approach - DEV Community, accessed February 8, 2026, https://dev.to/aragossa/stop-leaking-api-keys-in-your-ai-agent-logs-a-go-sidecar-approach-d3
13. Agent Calculus: A Unified Framework for AI Agent Design - DEV Community, accessed February 8, 2026, https://dev.to/herrington_darkholme/agent-calculus-a-unified-framework-for-ai-agent-design-32d8
14. Effective context engineering for AI agents - Anthropic, accessed February 8, 2026, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
15. The Agentic AI Security Scoping Matrix: A framework for securing autonomous AI systems, accessed February 8, 2026, https://aws.amazon.com/blogs/security/the-agentic-ai-security-scoping-matrix-a-framework-for-securing-autonomous-ai-systems/
16. Towards the Holographic Characteristic of LLMs for Efficient Short-text Generation - arXiv, accessed February 8, 2026, https://arxiv.org/html/2601.22546v1
17. Towards the Holographic Characteristic of LLMs for Efficient Short-text Generation - arXiv, accessed February 8, 2026, https://www.arxiv.org/pdf/2601.22546
18. Whiteboard Architecture for the Multi-agent Sensor Systems - Digikogu, accessed February 8, 2026, https://digikogu.taltech.ee/et/Download/efa6833b-f94e-43fd-a156-7c762348ab79/WhiteboardArchitecturefortheMultiagentSensor.pdf
19. CVPR Poster Embodied Multi-Modal Agent trained by an LLM from a Parallel TextWorld, accessed February 8, 2026, https://cvpr.thecvf.com/virtual/2024/poster/30385
20. A Survey of Continual Reinforcement Learning - arXiv, accessed February 8, 2026, https://arxiv.org/html/2506.21872v1
21. ICML Poster Continual Reinforcement Learning by Planning with Online World Models, accessed February 8, 2026, https://icml.cc/virtual/2025/poster/44151
22. Protocol Deviations in Clinical Trials and the Role of AI - Readout, accessed February 8, 2026, https://readout.ai/protocol-deviations-in-clinical-trials-and-the-role-of-ai/
23. Managing Protocol Deviations: A Guide for Clinical Trials - IntuitionLabs, accessed February 8, 2026, https://intuitionlabs.ai/articles/managing-protocol-deviations